Nama : Nurma Ayu Mulya Harianti
Kelas : 1-G
No. Presensi : 18

Percobaan 1

```
Masukkan nilai n = 5
*****
```

1. Jika inisialisasi i = 1 diubah menjadi i = 0 tanpa mengubah kondisi, berapa karakter * yang tercetak ketika n = 5? Jelaskan.
   Jawab: ada 6 karakter * yang tercetak. Hal ini, terjadi karena kondisi nya i <= n sehingga i memiliki 6 nilai yakni 0,1,2,3,4,5.

2. Jika kondisi i <= n diganti menjadi i < n, berapa kali perulangan dijalankan saat n = 5? Mengapa hasilnya berbeda?
   Jawab: hanya 4 karakter * yang tercetak. Kondisi i < n berati perulangan berjalan n-1 pada soal berati 5-1=4 sehingga i memiliki nilai 1,2,3,4.

3. Prediksi output potongan kode berikut ketika n = 9, kemudian buktikan dengan mengganti perulangan FOR pada program Anda dan menjalankannya. for (int i = 2; i <= n; i += 2) { System.out.print(i + " "); } Modifikasi kode tersebut agar mencetak bilangan ganjil positif yang tidak melebihi n.
   Jawab: jika kode for (int i = 2; i <= n; i += 2) { System.out.print(i + " "); } dijalankan terminal akan memunculkan bilangan genap positif dengan batas memodifikasi kode tersebut agar memiliki nilai bilangan ganjil positif adalah mengganti deklarasi int i =1.

```
for (int i = 1; i <=  n; i++){
    System.out.print("*");
for (int i = 1; i <=  n; i+=2){
    System.out.print(i + " ");
```

4. Jelaskan mengapa perubahan increment i++ menjadi i-- (dengan kondisi tetap i <= n) membuat perulangan berjalan terus-menerus dan tidak berhenti dalam waktu wajar untuk n positif. Jangan menjalankan versi tersebut; jika terlanjur berjalan, hentikan program dengan menekan Ctrl + C pada terminal.
   Jawab: Dekremen terus mengurangi nilai karna tidak dibatasi oleh kondisi sehingga kondisi selalu bernilai true. Hal ini lah yang membuat perulangan berjalan terus-menerus dan tidak berhenti dalam waktu wajar hingga menyentuh bilangan negatif.

Nama : Nurma Ayu Mulya Harianti
Kelas : 1-G
No. Presensi : 18

Percobaan 2

```
Masukkan jumlah baris: 3
Masukkan jumlah kolom: 6
******
```

```
Masukkan jumlah baris: 3
Masukkan jumlah kolom: 6
******
******
******
```

1. Tentukan fungsi variabel i dan j pada program tersebut.
   Jawab: variabel i penghitung baris (outer loop) dan j penghitug kolom (inner loop)

2. Berapa kali outer loop, inner loop, dan perintah print("*") dijalankan untuk baris = 4 dan kolom = 7?
   Jawab: 28 kali. Karna outer setiap outer loop mengeprint 7x inner loop sehingga 4x7=28 kali print “*”

3. Apa yang terjadi jika batas perulangan luar dan dalam ditukar (outer menggunakan kolom, inner menggunakan baris)? Gunakan baris = 3 dan kolom = 6 untuk menjelaskan.
   Jawab: outer akan berjalan 6 kali dan inner akan berjalan 3 kali sehingga akan tercetak 6 baris dan 3 kolom. namun yang berubah hanya bentuknya saja jumlah karakter * yang di print tetap 18.

```
Masukkan jumlah baris: 3
Masukkan jumlah kolom: 6
***
***
***
***
***
***
```

4. Jelaskan akibat penghapusan System.out.println(); setelah inner loop.
   Jawab: jika tidak menggunakan println() setelah inner loop teks tidak akan berpindah baris hingga semua bkarakter * yang di print akan menjadi satu deret.

```
Masukkan jumlah baris: 3
Masukkan jumlah kolom: 6
******************
```

5. Modifikasi program agar karakter yang dicetak adalah #, lalu commit dan push perubahan ke GitHub.

Percobaan 3

Simpan dan jalankan program dengan n = 3, kemudian dengan n = 5. Catat output aktual yang muncul, lalu bandingkan dengan output target.

```
Masukkan nilai n = 3
******
```

```
Masukkan nilai n = 5
***************
```

Jalankan kembali program dengan n = 5, lalu amati hasilnya:

```
Masukkan nilai n = 5
[i=0] [i=1] *[i=2] **[i=3] ***[i=4] ****[i=5] *****
```

a. berapa kali outer loop dijalankan? Apakah sama dengan jumlah baris yang diinginkan?
   outer loop berjalan 6 kali (i = 0, 1, 2, 3, 4, 5). jumlah baris Tidak sama dengan yang diinginkan (5 baris).
b. berapa karakter * yang tercetak saat i = 0?
   0 bintang/tidak ada
c. mengapa semua karakter tampil dalam satu baris?
   Karna tidak ada System.out.println() yang memindahkan teks ke baris baru setelah inner loop.

Perbaiki kode satu per satu dan jalankan program setelah setiap perbaikan:

a. tambahkan System.out.println(); setelah inner loop selesai, sebelum i++. Jalankan dan amati: apakah masih ada baris yang tidak berisi *?
   Ada, baris pertama

```
Masukkan nilai n = 5
[i=0]
[i=1] *
[i=2] **
[i=3] ***
[i=4] ****
[i=5] *****
```

b. ubah inisialisasi dan/atau kondisi outer loop agar perulangan berjalan tepat n kali tanpa baris kosong. Jalankan kembali.

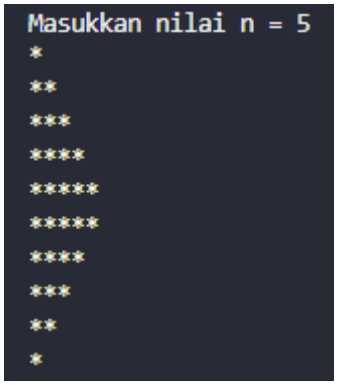
```
Masukkan nilai n = 5
[i=1] *
[i=2] **
[i=3] ***
[i=4] ****
[i=5] *****
```

Setelah output sama dengan output target, hapus baris debug. Jalankan sekali lagi untuk memastikan hasilnya,

```
Masukkan nilai n = 5
*
**
***
****
*****
```

Nama : Nurma Ayu Mulya Harianti
Kelas : 1-G
No. Presensi : 18

Modifikasi program agar setelah segitiga di atas tercetak, program juga mencetak segitiga terbalik. Tuliskan nested WHILE kedua di bawah nested WHILE pertama (masih di dalam main()), tanpa mengganti WHILE menjadi FOR.



1. Sebutkan ketidaksesuaian output awal dan minimal dua perbaikan yang diperlukan untuk menghasilkan pola segitiga bertingkat.
   Jawab: 1) bintang tercetak dalam satu deret. Perbaikan dengan menambahkan System.out.println(); pada outer loop sebelum update agar berpindah baris
   2) outer loop berjalan n+1 karna inisialisasi i=0. Perbaikan pengubah inisialisasi i =1 atau tetap menggunakan inisialisasi i=0 namun ubah j <= i.

2. Jelaskan peran variabel i dan j. Mengapa nilai j harus diinisialisasi kembali ke 0 pada setiap iterasi outer loop? Apa yang terjadi jika j tidak di-reset?
   Jawab: i berfungsi penghitung baris yang muncul dan j berfungsi sebagai penghitung kolom. j harus kembali ke 0 lagi di setiap iterasi outer loop karena setiap baris harus mulai menghitung bintangnya dari nol. Jika tidak di-reset nilai j sisa baris sebelumnya akan terbawa. Pada baris berikutnya i bertambah 1, sehingga kondisi j < i hanya benar satu kali dan hanya 1 bintang yang tercetak per baris. Hasilnya bukan segitiga, melainkan deretan satu bintang per baris. Contohnya n = 5 akan menampilkan total 5 bintang, bukan 15.

3. Jelaskan perubahan inisialisasi, kondisi, dan perubahan nilai (update) pada outer loop yang diperlukan untuk membentuk segitiga terbalik.
   Jawab: pada segitiga bertingkat : inisialisasi i = 1; kondisi i <= n; update i++ sedangkan pada segitiga terbalik: inisialisasi i = n; kondisi i >= 1; update i--.

Nama : Nurma Ayu Mulya Harianti
Kelas : 1-G
No. Presensi : 18

Percobaan 4

```
Masukkan jumlah kelompok : 3
Masukkan jumlah penilai  : 2
Kelompok 1
  Nilai dari Penilai 1: 80
  Nilai dari Penilai 2: 90
Kelompok 1: nilai rata-rata = 85.0
Kelompok 2
  Nilai dari Penilai 1: 75
  Nilai dari Penilai 2: 105
  Nilai harus 0-100. Ulangi input!
  Nilai dari Penilai 2: 85
Kelompok 2: nilai rata-rata = 80.0
Kelompok 3
  Nilai dari Penilai 1: 90
  Nilai dari Penilai 2: 96
Kelompok 3: nilai rata-rata = 93.0
============================
Kelompok terbaik    : 3
Rata-rata tertinggi : 93.0
```

```
Masukkan jumlah kelompok : 2
Masukkan jumlah penilai  : 3
Kelompok 1
  Nilai dari Penilai 1: 90
  Nilai dari Penilai 2: 87
  Nilai dari Penilai 3: 84
Kelompok 1: nilai rata-rata = 87.0
Kelompok 2
  Nilai dari Penilai 1: 91
  Nilai dari Penilai 2: 89
  Nilai dari Penilai 3: 90
Kelompok 2: nilai rata-rata = 90.0
============================
Kelompok terbaik    : 2
Rata-rata tertinggi : 90.0
```

```
Masukkan jumlah kelompok : 0
Masukkan jumlah penilai  : 6
Jumlah kelompok dan penilai harus > 0.
```

1. Mengapa totalNilai perlu di-reset di dalam outer loop, tetapi rataTertinggi tidak boleh di-reset pada setiap kelompok?
   Jawab: totalNilai harus di-reset karena setiap kelompok dihitung dari nol. jika tidak, nilai kelompok sebelumnya ikut terjumlah. Sedangkan pada perhitungan rataTertinggi harus bertahan karena menyimpan rekor dari semua kelompok yang sudah diproses jika di-reset, pembanding akan hilang dan hasil akhir hanya mencerminkan kelompok terakhir.

2. Untuk 3 kelompok dan 2 penilai (tanpa input yang salah), berapa kali badan perulangan FOR dijalankan secara keseluruhan?
   Jawab: 3 kelompok × 2 penilai = 6 kali.

3. Apa yang terjadi jika totalNilai dideklarasikan sebagai int dan rata-rata dihitung menggunakan pembagian dua bilangan bulat? Berikan contoh nilai.
   Jawab: Jika totalNilai dengan type int dibagi menggunakan jumlahPenilai bertype int maka hasilnya pembagiannya adalah bilangan bulat dan angka desimal akan terpotong (tidak di bulatkan). Jadi hasilnya tidak akurat.

4. Mengapa rataTertinggi diberi nilai awal -1, bukan 0? Berikan contoh input yang membuat hasil salah apabila nilai awalnya 0.
   Jawab: karna nilai valid minimal 0. Jika rataTertinggi awal 0, rata-rata kelompok yang bernilai 0 tidak memenuhi rataNilai > rataTertinggi, sehingga kelompokTerbaik tetap 0 (tidak valid). Contohnya 2 kelompok dengan semua nilai 0 → "Kelompok terbaik : 0". Dengan -1, kelompok pertama pasti tercatat.

5. Program ini memiliki tiga tingkat perulangan. Sebutkan masing-masing jenis perulangan beserta perannya.
   Jawab: WHILE (luar) berfungsi mengulang per kelompok sebanyak jumlah kelompok. FOR (tengah) berfungsi mengulang per penilai sebanyak jumlah penilai dengan jumlah iterasi sudah diketahui. DO-WHILE (dalam) berfungsi meminta input nilai minimal sekali dan mengulang selama nilai di luar rentang 0–100.